

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_0	Input A must cover zero, max positive, max negative, and walking-ones patterns when reduction op on A is active.	Randomization under Assignment_2 constraints A toward MAXPOS, MAXNEG, ZERO and walking-ones patterns when red_op_A=1.	bins for ZERO, MAXPOS, MAXNEG, default, and walking-ones (3'b001, 3'b010, 3'b100) iff(red_op_A).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_1	Input B must cover zero, max positive, max negative, and walking-ones patterns when reduction op on B is active and A reduction is inactive.	Randomization under Assignment_2 constraints B toward walking-ones patterns when red_op_B=1 and red_op_A=0.	bins for ZERO, MAXPOS, MAXNEG, default, and walking-ones (3'b001, 3'b010, 3'b100) iff(red_op_B && !red_op_A).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_2	Opcode must cover all valid operations, the transitions in order, and never take invalid values.	Randomization under Assignment_2 with weighted dist across all opcodes including INVALID_6/7 at lower weight.	bins for shift {SHIFT,ROTATE}, arith {ADD,MULT}, bitwise {OR,XOR}, illegal_bins for {INVALID_6,INVALID_7}, transition bin (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_3	All inputs are randomized with weighted distributions: opcodes equally weighted, bypass and rst favoring deasserted, A/B biased toward boundary values for arithmetic, walking-ones for bitwise, and direction biased for shift/rotate.	Assignment_2 constraint drives dist on opcode, bypass_A/B, rst, A, B, red_op_A/B, direction conditioned on opcode group.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_4	The ALSU must produce correct outputs for all 6 valid opcodes applied in a unique, non-repeating sequence.	Assignment_3 constraint forces my_array to hold all 6 opcodes (OR to ROTATE) uniquely using unique{my_array} and foreach range constraint.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_5	On reset assertion, outputs (out and leds) must be low.	Directed stimulus: rst forced to 1 then 0 twice using assert_reset task before any randomization begins.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_6	All ALSU operations produce correct outputs under fully randomized constrained stimulus across 320 iterations.	320-iteration loop with Assignment_2 constraints active and Assignment_3 constraints disabled. All inputs randomized and driven to DUT each iteration.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_7	All 6 valid opcodes exercised in a unique sequential order within each outer iteration, with bypass and reduction signals deasserted.	Assignment_3 constraints enabled after disabling all constraints. randomize() called once to fill my_array with 6 unique opcodes. rst, bypass, red_op signals forced to 0.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_8	All ALSU operations produce correct outputs under an extended 1000-iteration constrained random stimulus for deeper corner-case coverage.	1000-iteration loop with all constraints disabled except Assignment_2 constraints.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_9	The opcode transition coverage bin (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE) must be explicitly hit in the exact required order.	Obj.opcode set to OR then goes through XOR to ROTATE.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_10	When the ALU is addition or multiplication, A and B should have taken all permutations of maxpos, maxneg and zero	Constrained random	Cross bins: arithmetic op × {A_max, A_min, A_zero} × {B_max, B_min, B_zero} — 9 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_11	When the ALU is addition, c_in should have taken 0 or 1	Constrained random	Cross bins: ADD × {cin_0, cin_1} — 2 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_12	When the ALSU is shifting or rotating, then direction must take 0 or 1	Constrained random	Cross bins: shift/rotate op × {direction_0, direction_1} — 2 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_13	When the ALSU is shifting, then shift_in must take 0 or 1	Constrained random	Cross bins: SHIFT × {serial_in_0, serial_in_1} — 2 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_14	When the ALSU is OR or XOR and red_op_A is asserted, then A took all walking one patterns (001, 010, and 100) while B is taking the value 0	Constrained random	Cross bins: bitwise op × red_op_A=1 × {A_walkingones[0,1,2]} × B_zero — 3 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_15	When the ALSU is OR or XOR and red_op_B is asserted, then B took all walking one patterns (001, 010, and 100) while A is taking the value 0	Constrained random	Cross bins: bitwise op × red_op_B=1 × {B_walkingones[0,1,2]} × A_zero — 3 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_16	Covering the invalid case: reduction operation is activated while the opcode is not OR or XOR	Constrained random	Illegal bins: red_op_A=1 or red_op_B=1 with arithmetic or shift ops	A checker in the testbench to make sure the output is correct in comparison with a golden model.